

- Understand adversarial ML workflow
- Train a simple image classifier
- Generate FGSM adversarial examples
- Compare clean vs adversarial accuracy
- Focus on concepts, not memorizing syntax

torch → deep learning framework

torchvision → CIFAR-10 dataset + transforms

numpy → numerical operations

matplotlib → visualize predictions

IBM ART → adversarial robustness toolkit

You can Youtube or Google the videos to learn

Neural Networks in PyTorch: <https://youtu.be/c36lUUr864M>
(long video but good learning, you can watch at 2x speed and you can skip Familiar portions from your ML courses)

PyTorch Basics in Anaconda: <https://youtu.be/EMXfZB8FVUA>

PyTorch in Google Colab: <https://www.youtube.com/watch?v=vuNQo84OSfw>

FGSM Tutorial: <https://youtu.be/PFS9KQcQT-s>
(great revision after our class slides explanation)

IBM ART Toolkit: <https://youtu.be/9LMC90o4QxE>
(introduction from IBM)

Also learn how to load python libraries into Google Colab (lots of videos)
Pasted one here: <https://www.youtube.com/watch?v=N6J4weYZ9IU&t=18s>

You can load the ART library into Google Colab easily

Load Libraries

Hint: Ask Claude or Codex, what libraries you may need!

Load Dataset

Hint: use torchvision.datasets.CIFAR10

Use DataLoader for batching

Normalize image tensors

Use mini-batches

- Use pretrained ResNet-18
- Modify final classification layer
- Output should match 10 CIFAR-10 classes

Core workflow: prediction → loss → gradient → update

Use CrossEntropyLoss

Use optimizer.step()

Measure performance on normal images
Store predictions and labels
Compute classification accuracy
Question: is high clean accuracy enough?

Wrap PyTorch model using PyTorchClassifier
ART needs loss function + optimizer
This enables adversarial attacks

Use FastGradientMethod

Experiment with epsilon values

Small epsilon $\rightarrow$ subtle perturbation

Large epsilon $\rightarrow$ stronger attack

- Generate adversarial examples
- Compare clean vs adversarial accuracy
- Visualize image differences
- Create final comparison chart using visualization
- Vary Epsilon to see how the accuracy degrades
- Repeat cycle from above
- Reflect: why do humans still recognize the image?
- Refer Rubric and prepare submission:
 - Python Colab notebook
 - Results screenshot in PPTX format
 - Explanation/Reflection